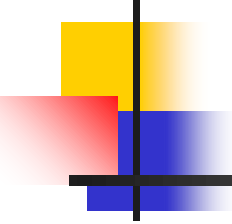
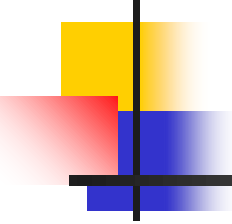


Regular Expressions



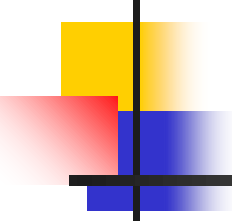
Regular Expression: Introduction

- *Regular expressions* are an algebraic way to describe languages.
- They describe exactly the regular languages.
- If E is a regular expression, then $L(E)$ is the language it defines.
- Offers a declarative way to express the pattern of any string we want to accept
 - E.g., $01^* + 10^*$



RE's: Definition

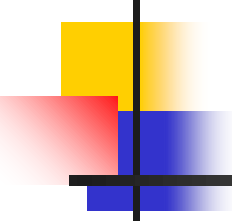
- **Basis 1:** If a is any symbol, then $\mathbf{a}$ is a RE, and $L(\mathbf{a}) = \{a\}$.
 - **Note:** $\{a\}$ is the language containing one string, and that string is of length 1.
- **Basis 2:** ε is a RE, and $L(\varepsilon) = \{\varepsilon\}$.
- **Basis 3:** $\emptyset$ is a RE, and $L(\emptyset) = \emptyset$.



RE's: Definition – (2)

- **Induction 1:** If E_1 and E_2 are regular expressions, then $E_1 + E_2$ is a regular expression, and $L(E_1 + E_2) = L(E_1) \cup L(E_2)$.
- **Induction 2:** If E_1 and E_2 are regular expressions, then $E_1 E_2$ is a regular expression, and $L(E_1 E_2) = L(E_1) L(E_2)$.

Concatenation : The set of strings wx such that w is in $L(E_1)$ and x is in $L(E_2)$.

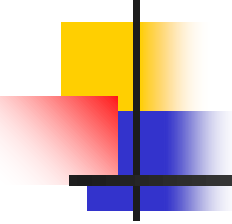


RE's: Definition – (3)

- **Induction 3:** If E is a RE, then E^* is a RE, and $L(E^*) = (L(E))^*$.

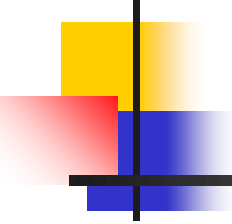
Closure, or “Kleene closure” = set of strings $w_1w_2\dots w_n$, for some $n \geq 0$, where each w_i is in $L(E)$.

Note: when $n=0$, the string is ε .



Precedence of Operators

- Parentheses may be used wherever needed to influence the grouping of operators in the regular expression.
- Order of precedence:
 - ✓ * (highest)
 - ✓ • concatenation
 - ✓ + (lowest)
- Example:
 - $01^* + 1 = (0 \cdot ((1)^*)) + 1$



Examples: RE's

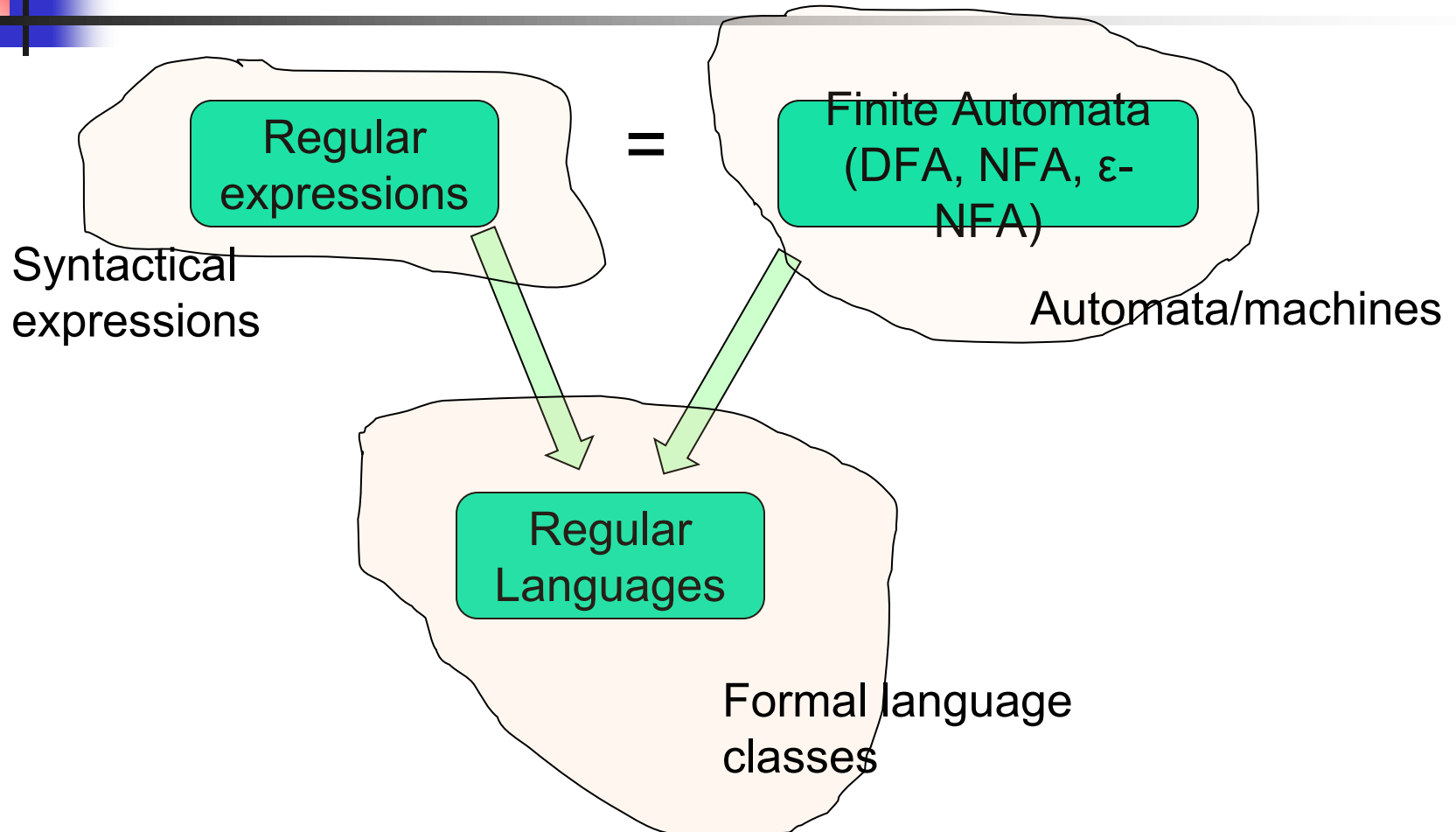
- $L(\mathbf{01}) = \{01\}$.
- $L(\mathbf{01+0}) = \{01, 0\}$.
- $L(\mathbf{0(1+0)}) = \{01, 00\}$.
 - Note order of precedence of operators.
- $L(\mathbf{0^*}) = \{\epsilon, 0, 00, 000, \dots\}$.
- $L(\mathbf{(0+10)^*(\epsilon+1)}) =$ all strings of 0's and 1's without two consecutive 1's.

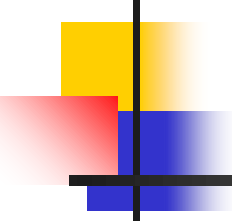


Equivalence of RE's and Automata

- We need to show that for every RE, there is an automaton that accepts the same language.
 - Pick the most powerful automaton type: the ϵ -NFA.
- And we need to show that for every automaton, there is a RE defining its language.
 - Pick the most restrictive type: the DFA.

Regular Expressions





Language Operators

- Union of two languages:
 - $L \cup M$ = all strings that are either in L or M
 - Note: A union of two languages produces a third language
- Concatenation of two languages:
 - $L . M$ = all strings that are of the form xy s.t., $x \in L$ and $y \in M$
 - The *dot* operator is usually omitted
 - i.e., LM is same as $L.M$

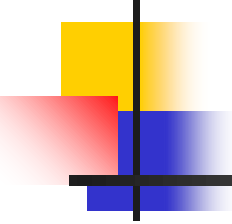
“i” here refers to how many strings to concatenate from the parent language L to produce strings in the language L^i

Kleene Closure (the * operator)

- Kleene Closure of a given language L:
 - $L^0 = \{\epsilon\}$
 - $L^1 = \{w \mid \text{for some } w \in L\}$
 - $L^2 = \{w_1 w_2 \mid w_1 \in L, w_2 \in L \text{ (duplicates allowed)}\}$
 - $L^i = \{w_1 w_2 \dots w_i \mid \text{all } w\text{'s chosen are } \in L \text{ (duplicates allowed)}\}$
 - (Note: the choice of each w_i is independent)
 - $L^* = \bigcup_{i \geq 0} L^i$ (arbitrary number of concatenations)

Example:

- Let $L = \{1, 00\}$
 - $L^0 = \{\epsilon\}$
 - $L^1 = \{1, 00\}$
 - $L^2 = \{11, 100, 001, 0000\}$
 - $L^3 = \{111, 1100, 1001, 10000, 000000, 00001, 00100, 0011\}$
 - $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$



Building Regular Expressions

- Let E be a regular expression and the language represented by E is $L(E)$
- Then:
 - $(E) = E$
 - $L(E + F) = L(E) \cup L(F)$
 - $L(E F) = L(E) L(F)$
 - $L(E^*) = (L(E))^*$

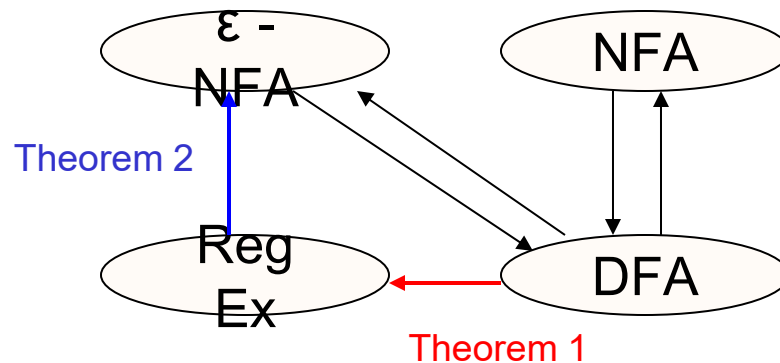
Example: how to use these regular expression properties and language operators?

- $L = \{ w \mid w \text{ is a binary string which does not contain two consecutive 0s or two consecutive 1s anywhere} \}$
 - E.g., $w = 01010101$ is in L , while $w = 10010$ is not in L
- Goal: Build a regular expression for L
- Four cases for w :
 - Case A: w starts with 0 and $|w|$ is even
 - Case B: w starts with 1 and $|w|$ is even
 - Case C: w starts with 0 and $|w|$ is odd
 - Case D: w starts with 1 and $|w|$ is odd
- Regular expression for the four cases:
 - Case A: $(01)^*$
 - Case B: $(10)^*$
 - Case C: $0(10)^*$
 - Case D: $1(01)^*$
- Since L is the union of all 4 cases:
 - Reg Exp for $L = (01)^* + (10)^* + 0(10)^* + 1(01)^*$
- If we introduce ϵ then the regular expression can be simplified to:
 - Reg Exp for $L = (\epsilon + 1)(01)^*(\epsilon + 0)$

Finite Automata (FA) & Regular Expressions (Reg Ex)

- To show that they are interchangeable, consider the following theorems:
 - Theorem 1: For every DFA A there exists a regular expression R such that $L(R)=L(A)$
 - Theorem 2: For every regular expression R there exists an ϵ -NFA E such that $L(E)=L(R)$

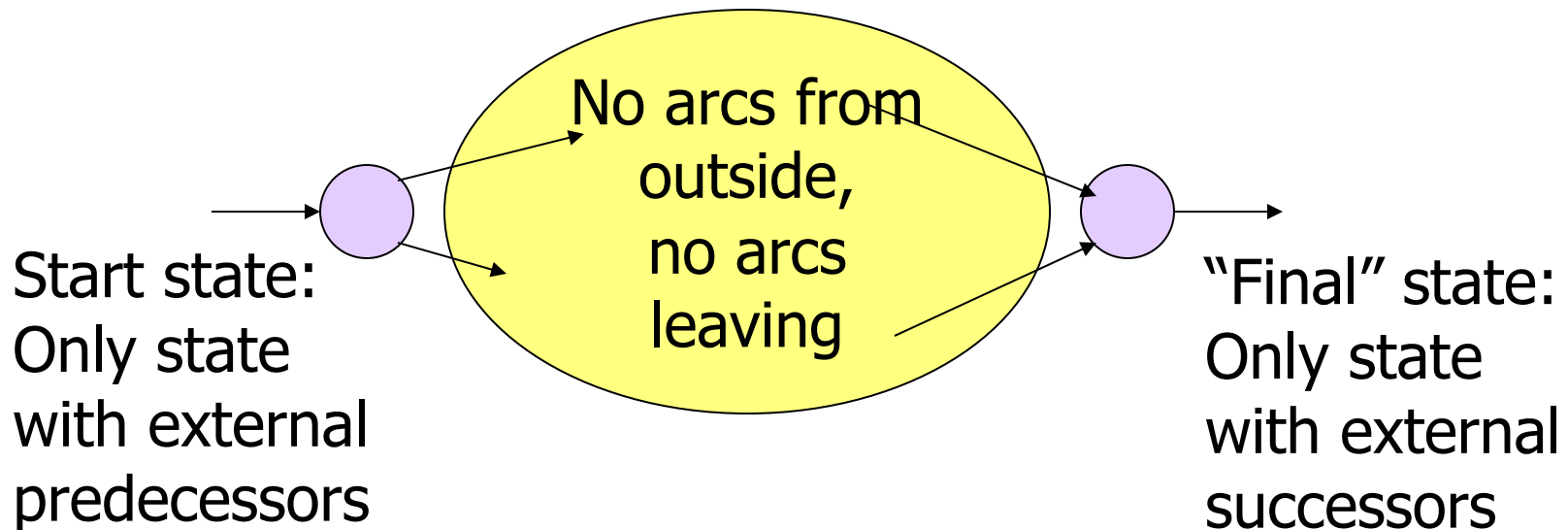
Proofs
in the book



Kleene Theorem

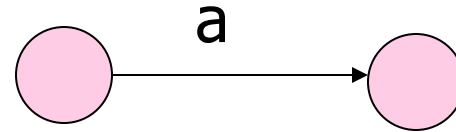
Converting a RE to an ε -NFA

□ Form of ε -NFA's Constructed

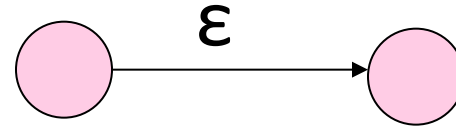


RE to ϵ -NFA: Basis

- Symbol **a**:



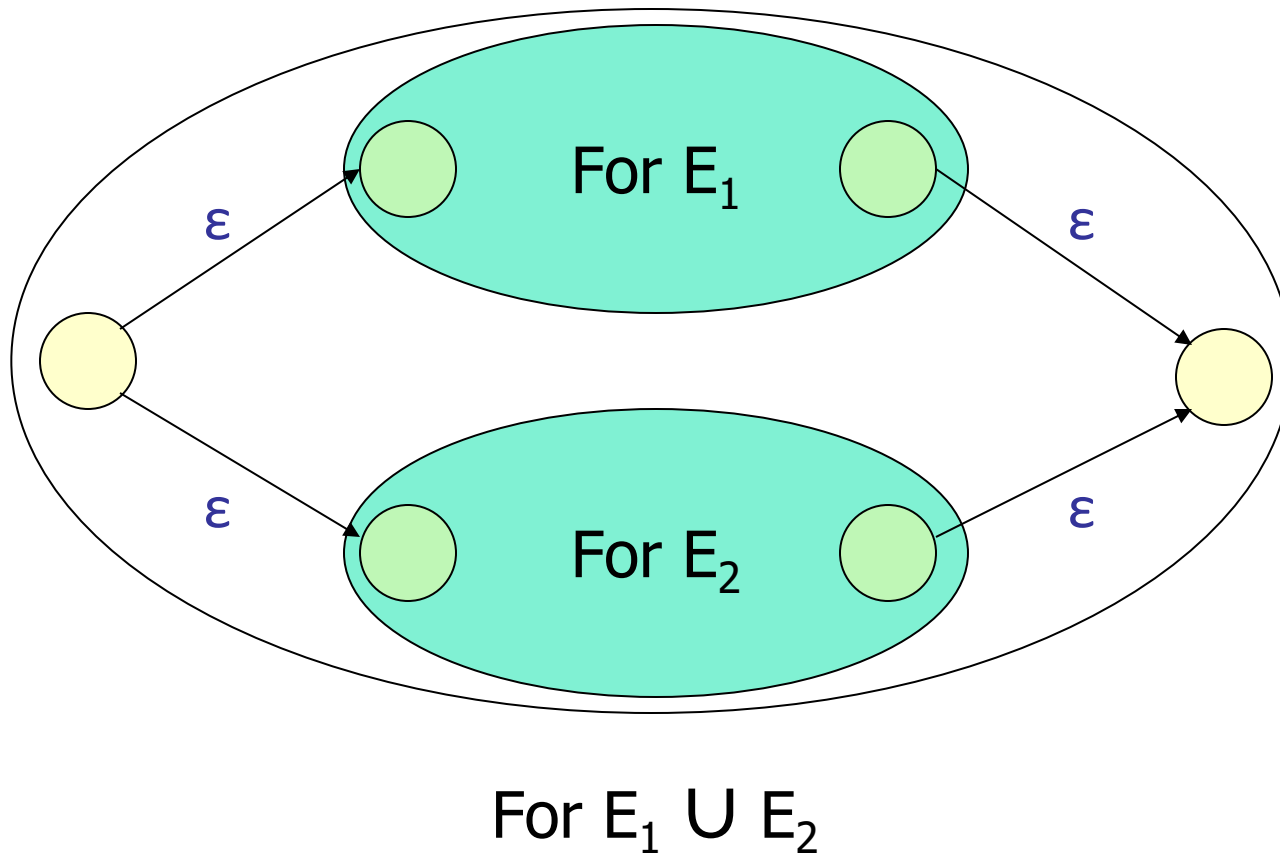
- ϵ :



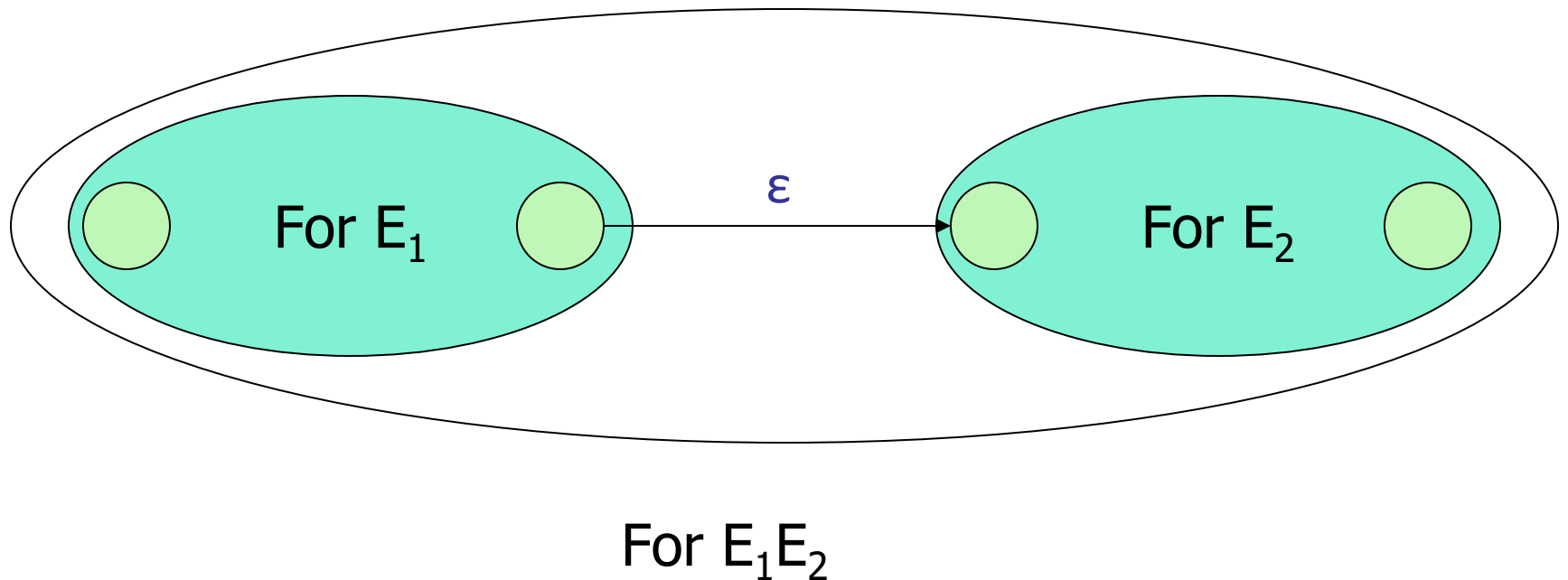
- $\emptyset$:



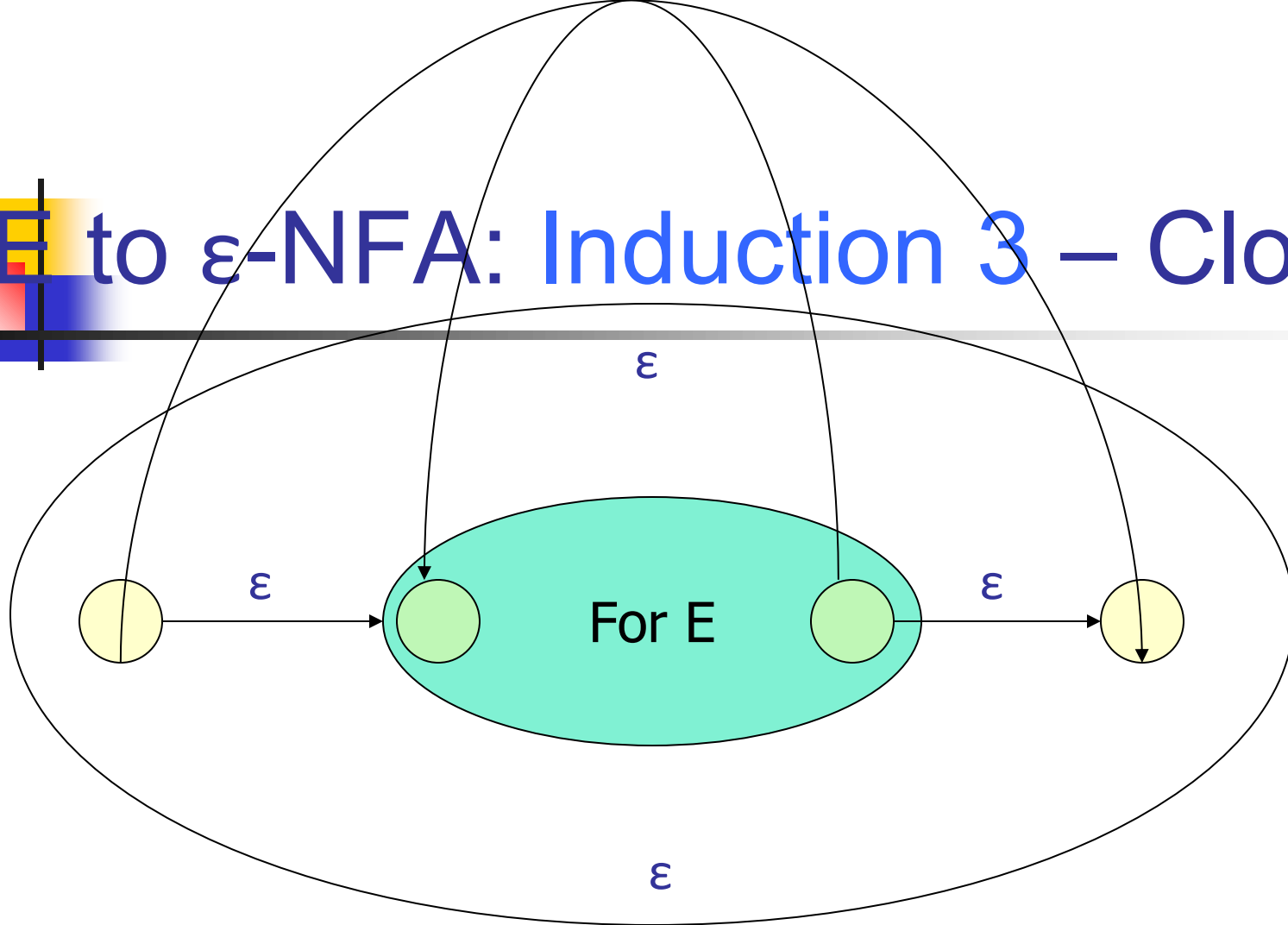
RE to ϵ -NFA: Induction 1 – Union



RE to ϵ -NFA: Induction 2 – Concatenation



RE to ϵ -NFA: Induction 3 – Closure



For E^*

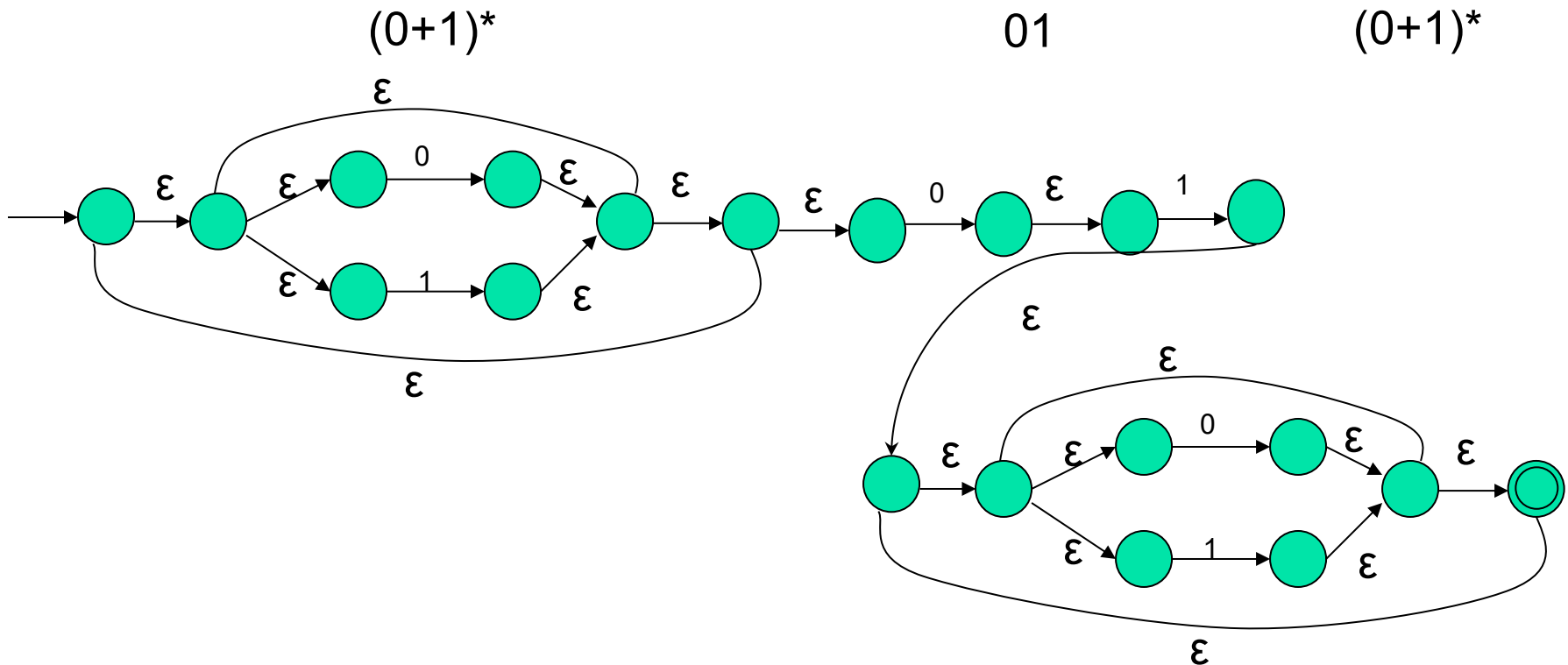
Reg Ex

Theorem 2

ϵ -NFA

RE to ϵ -NFA construction

Example: $(0+1)^*01(0+1)^*$



DFA

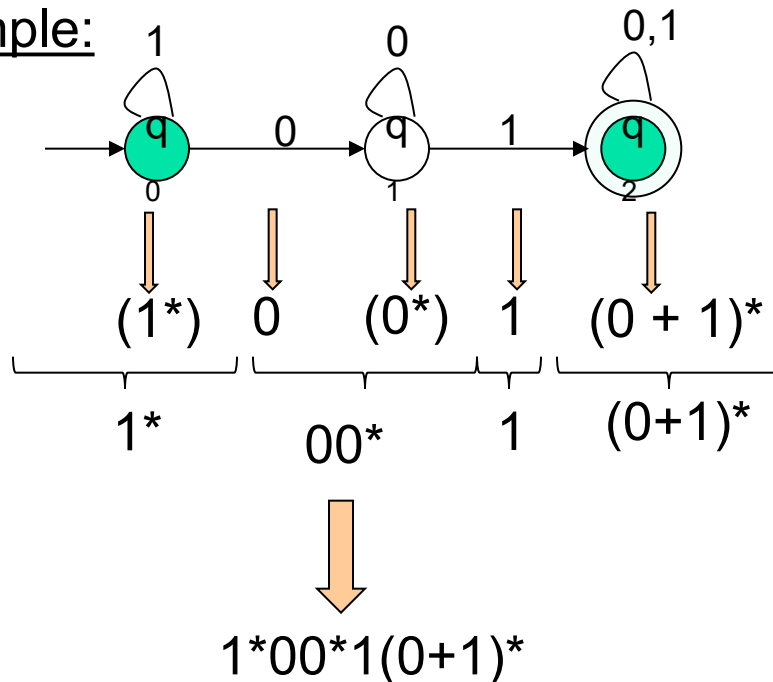
Theorem 1

Reg Ex

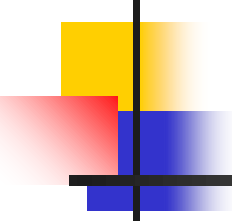
DFA to RE construction

Informally, trace all distinct paths (traversing cycles only once) from the start state to *each of the* final states and enumerate all the expressions along the way

Example:

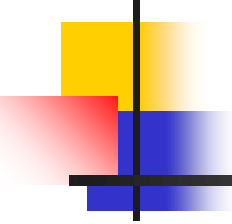


Q) What is the language?



DFA-to-RE

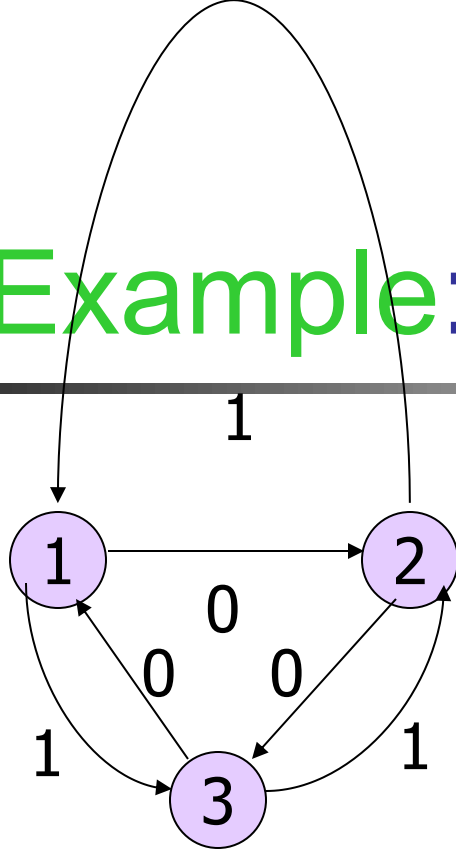
- A strange sort of induction.
- States of the DFA are assumed to be $1, 2, \dots, n$.
- We construct RE's for the labels of restricted sets of paths.
 - **Basis**: single arcs or no arc at all.
 - **Induction**: paths that are allowed to traverse next state in order.



k-Paths

- A k-path is a path through the graph of the DFA that goes **through** no state numbered higher than k.
- Endpoints are not restricted; they can be any state.

Example: k-Paths

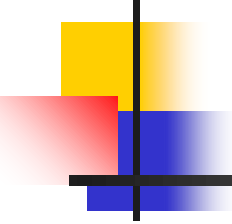


0-paths from 2 to 3:
RE for labels = **0**.

1-paths from 2 to 3:
RE for labels = **0+11**.

2-paths from 2 to 3:
RE for labels =
(10)*0+1(01)*1

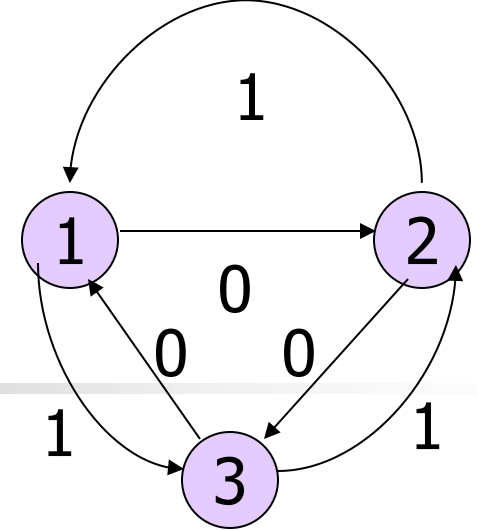
3-paths from 2 to 3:
RE for labels = ??



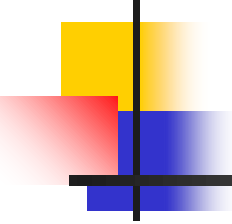
k-Path Induction

- Let R_{ij}^k be the regular expression for the set of labels of k-paths from state i to state j .
- **Basis:** $k=0$. R_{ij}^0 = sum of labels of arc from i to j .
 - $R_{ij}^0 = a$, if there is an arc a from i to j .
 - $R_{ij}^0 = a_1 + a_2 + \dots + a_k$, if there are k distinct arcs from i to j
 - $\emptyset$ if no such arc.
 - But add ε if $i=j$.

Example: Basis



- $R_{12}^0 = \mathbf{0}$.
- $R_{11}^0 = \emptyset + \varepsilon = \varepsilon$.



k-Path Inductive Case

- A k-path from i to j either:
 1. Never goes through state k, or
 2. Goes through k one or more times.

$$R_{ij}^k = R_{ij}^{k-1} + R_{ik}^{k-1} (R_{kk}^{k-1})^* R_{kj}^{k-1}.$$

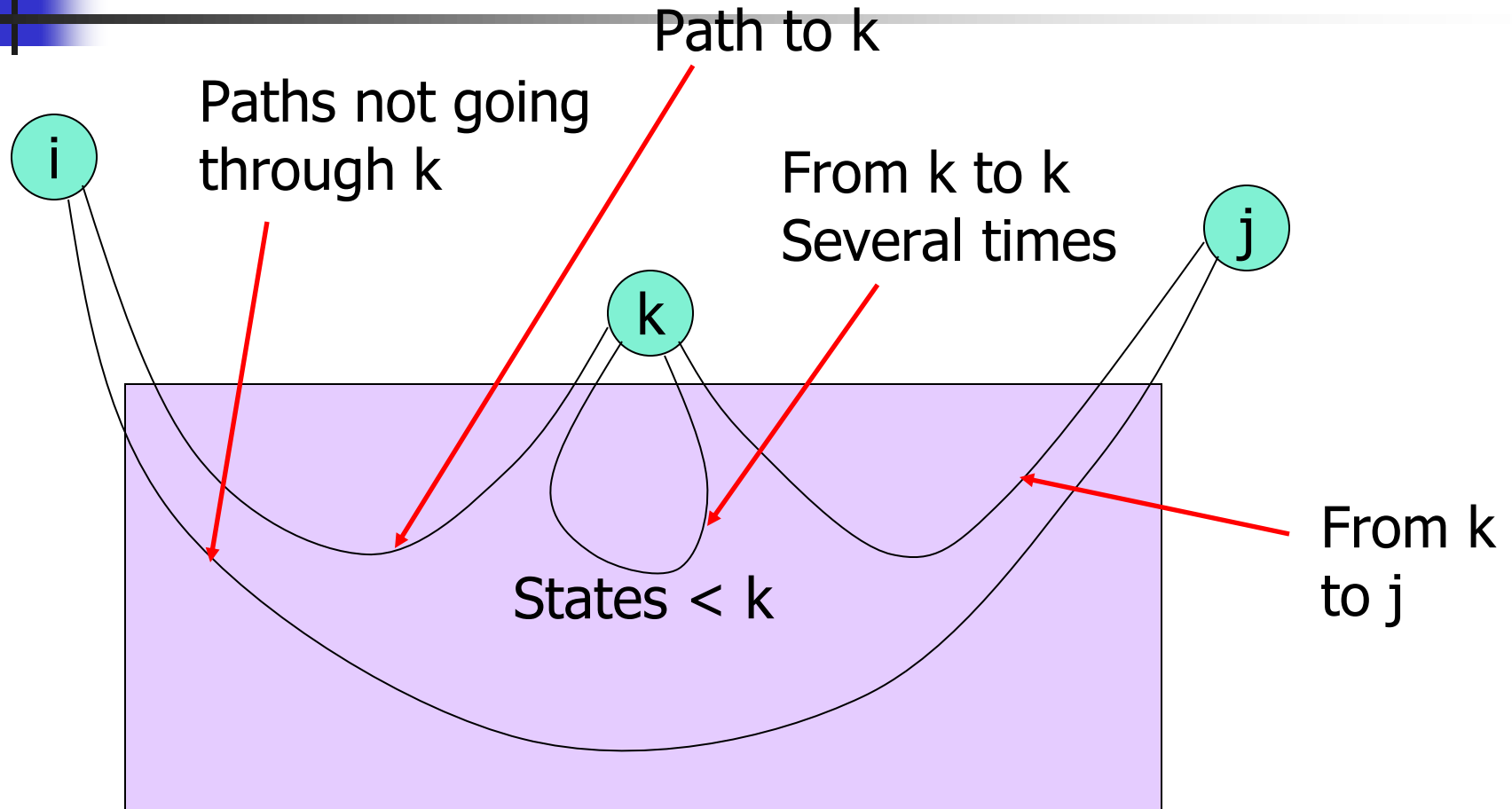
Doesn't go
through k

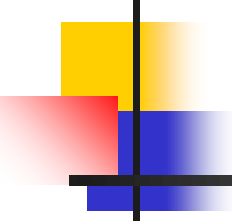
Goes from
i to k the
first time

Zero or
more times
from k to k

Then, from
k to j

Illustration of Induction

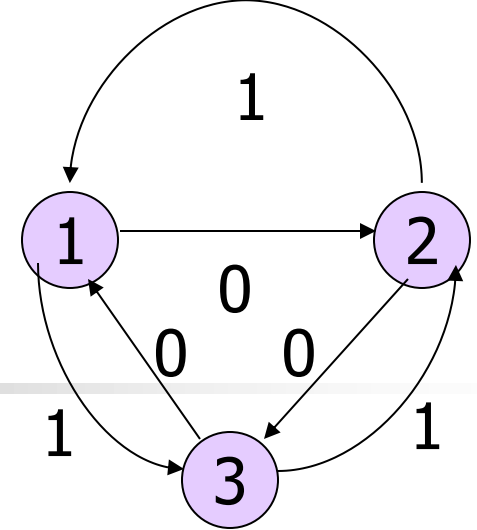




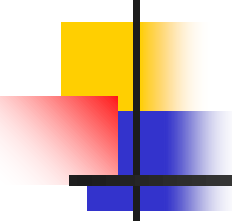
Final Step

- The RE with the same language as the DFA is the sum (union) of R_{ij}^n , where:
 1. n is the number of states; i.e., paths are unconstrained.
 2. i is the start state.
 3. j is one of the final states.

Example

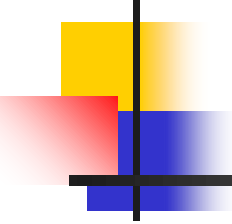


- $R_{23}^3 = R_{23}^2 + R_{23}^2(R_{33}^2)^*R_{33}^2 = R_{23}^2(R_{33}^2)^*$
- $R_{23}^2 = (10)^*0 + 1(01)^*1$
- $R_{33}^2 = 0(01)^*(1+00) + 1(10)^*(0+11)$
- $R_{23}^3 = [(10)^*0 + 1(01)^*1] [(0(01)^*(1+00) + 1(10)^*(0+11))]^*$



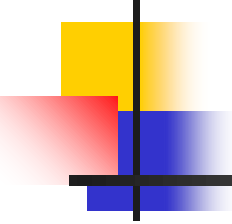
Summary

- Each of the three types of automata (DFA, NFA, ϵ -NFA) we discussed, and regular expressions as well, define exactly the same set of languages: the regular languages.



Algebraic Laws of Regular Expressions

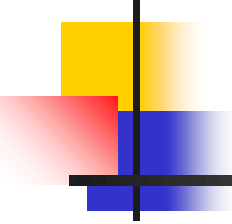
- Commutative:
 - $E + F = F + E$
- Associative:
 - $(E + F) + G = E + (F + G)$
 - $(EF)G = E(FG)$
- Identity:
 - $E + \Phi = E$
 - $\varepsilon E = E \varepsilon = E$
- Annihilator:
 - $\Phi E = E \Phi = \Phi$



Algebraic Laws...

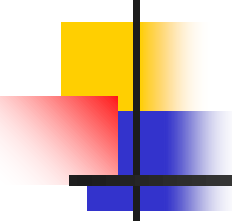
(Identities and Annihilators)

- $\emptyset$ is the identity for $+$.
 - $R + \emptyset = R$.
- ε is the identity for concatenation.
 - $\varepsilon R = R\varepsilon = R$.
- $\emptyset$ is the annihilator for concatenation.
 - $\emptyset R = R\emptyset = \emptyset$.



Algebraic Laws...

- Distributive:
 - $E(F+G) = EF + EG$
 - $(F+G)E = FE+GE$
- Idempotent: $E + E = E$
- Involving Kleene closures:
 - $(E^*)^* = E^*$
 - $\Phi^* = \varepsilon$
 - $\varepsilon^* = \varepsilon$
 - $E^+ = EE^*$
 - $E? = \varepsilon + E$



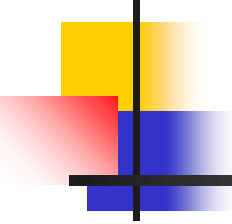
Algebraic Laws...

- Theorem-3.11: if L , M , and N are any languages, then $L(M \cup N) = LM \cup LN$.

Proof:

If $w \in L(M \cup N)$ then $w = xy$, where $x \in L$ and $y \in M$ or $y \in N$.
If $y \in M$, then $xy \in LM$, therefore $xy \in LM \cup LN$. Likewise, If $y \in N$, then $xy \in LN$, therefore $xy \in LM \cup LN$.

Again, suppose $w \in LM \cup LN$. Then $w \in LM$ or $w \in LN$. If $w = xy$, then $x \in L$ and $y \in M$ or $y \in N$. If $y \in M$, then $y \in M \cup N$. Therefore, $w \in L(M \cup N)$. Otherwise, $y \in N$ and $y \in M \cup N$. Therefore, $w \in L(M \cup N)$.



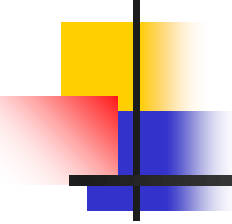
True or False?

Let R and S be two regular expressions. Then:

1. $((R^*)^*)^* = R^*$?

1. $(R+S)^* = R^* + S^*$?

1. $(RS + R)^* RS = (RR^*S)^*$?



Summary

- Regular expressions
- Equivalence to finite automata
- DFA to regular expression conversion
- Regular expression to ϵ -NFA conversion
- Algebraic laws of regular expressions
- Unix regular expressions and Lexical Analyzer